

Assignment 2

Question 1

1. Scenario and Objective

Modern runtime systems maintain a call stack to manage active function invocations. Each invocation creates an activation record (stack frame) containing execution state and occupying a finite amount of stack memory. Recursive calls create independent frames, nested calls temporarily suspend their callers, and an attempted allocation may fail when the remaining stack memory is insufficient. You are required to design a C++ simulator for this execution model. The simulator must use a student-implemented stack backed by a dynamic array. It must process multiple independent test cases from a text file, evaluate recursion counts, simulate depth-first execution, enforce memory alignment, detect stack overflow, and produce a detailed execution trace and summary.

2. Core Restrictions

- The call stack must be implemented using a dynamic array. Do not use built in stack.
- All test cases are read from a single text file.
- Arithmetic and ternary expressions in the supplied input may be assumed syntactically valid; expression-syntax validation is not required.

3. Function Definitions

Each function definition has the following conceptual form:

`<function_name>(<recursion_count>){<nested_calls>} <requested_memory>`

The recursion count and nested-call list are optional. Examples:

`funA(3){funB,(true?Dummy:funD)} 5`

`funB(){}` 7

`Dummy(12/6){funE} 9`

`funD(){}` 4

`funE(){}` 6

4. Function Name Rules

- A function name must begin with an alphabetic character.
- After the first character, only alphabetic characters and digits are permitted.
- A function can be defined only once within a test case.

5. Function Calls and Stack Frames

When a function invocation begins, its stack frame is pushed onto the call stack. When that invocation finishes, the frame is popped. A nested function must finish completely before its caller resumes.

funB called

funB finished

Each invocation is independent. Therefore, recursive invocations of the same function occupy separate stack frames and consume stack memory separately.

6. Recursion

A recursion count can be a positive integer or an arithmetic expression using +, -, *, / and parentheses. The supplied arithmetic expression may be assumed syntactically valid. Its evaluated result must be used as the recursion count.

Function1(3)

Function2(12/6)

Function3(2+3*2)

For a recursive function, recursion occurs before the nested-call body of the current invocation. After the deepest invocation executes its nested calls and finishes, control returns to the previous invocation, which then executes its own nested calls.

For example, for:

funA(3){funB}

the required execution order is:

funA called

funA called

funA called

funB called

funB finished

funA finished

funB called

funB finished

funA finished

funB called

funB finished

funA finished

Thus, every recursive invocation owns and executes its own nested-call body.

7. Nested Calls

Functions may contain zero or more nested calls inside braces. Nested calls are processed from left to right and execute depth-first.

funA(){funB,funC} 5

funB(){}

funC(){}

Calling funA results in:

funA called
funB called
funB finished
funC called
funC finished
funA finished

8. Conditional Nested Calls Using the Ternary Operator

A nested-call position may contain a ternary conditional. The condition is restricted to the literal values true and false, and each complete ternary segment must be enclosed in parentheses.

(true?funB:funC)
(false?funD:funE)

- If the condition is true, only the first branch is executed.
- If the condition is false, only the second branch is executed.
- The unselected branch is not called, is not pushed, consumes no stack memory, and does not contribute to runtime call statistics.

Example:

funA(2){(true?funB:funC),funD,(false?funE:funF)} 5

For each invocation of funA, the selected nested-call sequence is funB, funD, funF. Ternary syntax may be assumed valid; you are responsible for interpreting it, not validating it.

9. Memory-Based Stack Capacity

The stack limit S represents total stack memory in bytes, not the maximum number of entries. Every function specifies the requested memory for one invocation.

funA(){ } 5
funB(){ } 12

Memory remains allocated only while the corresponding invocation is active. When a frame is popped, all memory allocated to that frame is released.

10. Four-Byte Stack Frame Alignment

The runtime uses 4-byte stack-frame alignment. A function may request any positive integer number of bytes, but its actual allocation must be the smallest multiple of 4 greater than or equal to the requested size.

Requested Memory	Allocated Memory
1 B	4 B
3 B	4 B

4 B	4 B
5 B	8 B
7 B	8 B
8 B	8 B
9 B	12 B
13 B	16 B
17 B	20 B

Overflow calculations must use allocated memory, not requested memory. The stack capacity S itself is not required to be a multiple of 4.

11. Stack Overflow

Before pushing a frame, determine its aligned allocation and compare it with the currently available stack memory. If the frame does not fit, it must not be pushed.

Error: Stack overflow while calling <function_name>

The failed invocation and its entire body are skipped. If it was a nested call, its caller remains active and continues with the next nested operation. A stack overflow is a runtime error and does not terminate the entire test case.

Example:

Stack capacity: 20 B

funA requests 5 B -> allocates 8 B

funB requests 7 B -> allocates 8 B

Current use after A and B: 16 B

funC requests 5 B -> requires 8 B

$16 + 8 = 24 \text{ B} > 20 \text{ B}$

Error: Stack overflow while calling funC

12. Static Dependency Validation

Before executing any top-level call, the complete set of function definitions in that test case must be analyzed. Static validation considers all nested references, including both branches of a ternary expression, regardless of which branch would execute at runtime.

12.1 Undefined Functions

Every referenced function must have a definition in the same test case. This includes functions inside unselected ternary branches and top-level calls.

Error: Undefined function funXYZ called by funA

Error: Undefined top-level function funXYZ

12.2 Duplicate Definitions

If a function is defined more than once within a test case, report:

Error: Duplicate definition of funA

Do not silently overwrite an earlier definition.

12.3 Circular Dependencies

Circular dependency checking is global: all defined functions must be checked before execution, even if a cyclic component is unreachable from the supplied top-level calls.

```
funA(){funB,funC} 4
```

```
funB(){funD} 5
```

```
funC(){funE} 7
```

```
funD(){} 8
```

```
funE(){funF} 4
```

```
funF(){funC} 9
```

The dependency cycle is:

```
funC -> funE -> funF -> funC
```

The program must print the actual discovered cycle:

```
Error: Circular Dependency: funC -> funE -> funF -> funC
```

If multiple cycles exist, report only the first cycle discovered according to function-definition order and nested-call order.

13. Static Errors vs Runtime Errors

Duplicate definitions, undefined function references, and circular dependencies are static errors. If any such error is found, execution of that test case must not begin. Stack overflow is a runtime error; only the failed invocation is skipped and the test case continues.

14. Stack Status Display

After every successful push and pop, display the current stack. Each entry must show enough information to identify the active function and its memory allocation. Also display used and available stack memory.

```
funB called
```

```
Stack: [funA:8] -> [funB:8] <- TOP
```

```
Memory: 16/32 B
```

15. Execution Statistics

After a test case completes execution, print an execution summary. A call attempt is counted when execution attempts to invoke a selected function. Calls rejected because of stack overflow count as attempts but not successful calls. Unselected ternary branches do not count as attempts.

```
Execution Summary
```

```
-----
```

```
Total call attempts: 21
```

Successful calls: 18

Skipped due to stack overflow: 3

Maximum stack depth reached: 6

Maximum stack memory used: 44 B

Total stack capacity: 48 B

Most frequently called function: funB

For 'Most frequently called function', count successful invocations. If multiple functions tie, report the one whose definition appears first in the test case.

16. Input File and Multiple Test Cases

You may use AI tools to implement this part. No questions will be asked regarding this functionality during the demo. However, this functionality is **mandatory**, and failure to implement it will result in **negative marking**.

Input must be read from a text file (for example, input.txt). A single file may contain multiple independent test cases. Test cases are separated by a line containing exactly:

###

The first line of each test case is:

N M S

- N = number of function definitions.
- M = number of top-level function calls.
- S = total call-stack capacity in bytes.

The next N lines contain function definitions. The following M lines contain top-level function calls, which execute sequentially in the order listed.

Example input file:

3 2 32

funA(3){funB} 5

funB(){funC} 7

funC(){}

funA

funC

###

4 1 48

main(2){load,(true?process:save)} 7

```
load({}) 5
process(3){save} 9
save({}) 4
main
```

###

```
3 1 20
funA(){funB} 5
funB(){funC} 7
funC(){funA} 4
funA
```

When a separator is reached, all state belonging to the previous test case must be discarded. Definitions, stack contents, memory usage, dependency-analysis state, counters, and statistics must not leak into the next test case. Blank lines may be ignored.

17. Required Processing Order Per Test Case

1. Read N, M, and S.
2. Read all N function definitions.
3. Read all M top-level calls.
4. Check function-name rules and duplicate definitions.
5. Check all referenced functions, including both ternary branches and top-level calls.
6. Build/analyze the complete dependency structure and detect circular dependencies. If a static error exists, report it and skip runtime execution for that test case.
7. Evaluate recursion-count expressions as needed.
8. Execute top-level calls sequentially using the memory-aligned call stack.
9. For each selected invocation, perform the memory-allocation check before pushing its frame.
10. Display call/finish/error messages and stack status after each successful push/pop.
11. After runtime execution, print the execution summary.

18. Required Output Messages

```
<function_name> called
<function_name> finished
Error: Stack overflow while calling <function_name>
Error: Circular Dependency: <cycle path>
Error: Undefined function <name> called by <caller>
Error: Undefined top-level function <name>
Error: Duplicate definition of <function_name>
```

For multiple test cases, clearly identify each case in the console output, for example:

===== Test Case 1 =====

19. Constraints

- $1 \leq N \leq 50$
- $1 \leq M \leq 50$
- S is a positive integer number of bytes.
- Requested function memory is a positive integer number of bytes.
- Recursion expressions use integer arithmetic with $+$, $-$, $*$, $/$ and parentheses and are assumed syntactically valid.
- Conditions in ternary nested calls are restricted to true and false.
- Input may contain multiple test cases separated by ###.

20. Implementation Requirements

- Implement the call stack using a dynamic array; `std::stack` is not permitted.
- Maintain current used memory and compute aligned frame allocation before each push.
- Represent function definitions so nested-call order and both ternary branches can be analyzed.
- Perform global circular-dependency analysis and reconstruct the reported cycle.
- Process every test case in the input text file independently.
- Display the complete execution trace, stack state, memory state, errors, and required statistics on the console.

21. Illustrative Memory Example

Input fragment:

Stack capacity $S = 31$ B

`funA(3){funB}` 7

`funB(){}` 5

Aligned sizes:

`funA`: 7 B requested -> 8 B allocated

`funB`: 5 B requested -> 8 B allocated

At deepest `funA` recursion:

$3 \times 8 \text{ B} = 24 \text{ B}$ used

7 B available

Attempting `funB` requires 8 B, therefore:

Error: Stack overflow while calling `funB`

The failed funB is skipped. The deepest funA finishes, releases 8 B,
and the previous funA invocation then attempts its own funB call.

This example illustrates why overflow must be evaluated dynamically for every invocation: the same nested function may fail at one recursion depth and succeed later after stack memory has been released.

Sample Test Case

INPUT FILE:

```
4 1 28
funA(2){funB,(true?funC:funD)} 5
funB(){ } 7
funC(){ } 9
funD(){ } 4
funA
```

###

```
3 1 20
main(2){work} 5
work(){help} 7
help(){ } 5
main
```

###

```
3 1 32
funA(){funB} 4
funB(){funC} 4
funC(){funA} 4
funA
```

###

```
2 1 24
start(){worker,missing} 5
worker(){ } 4
```

start

###

2 1 24

task({}) 5

task({}) 9

task

OUTPUT:

===== Test Case 1 =====

funA called

Stack: [funA:8] <- TOP

Memory: 8/28 B

funA called

Stack: [funA:8] -> [funA:8] <- TOP

Memory: 16/28 B

funB called

Stack: [funA:8] -> [funA:8] -> [funB:8] <- TOP

Memory: 24/28 B

funB finished

Stack: [funA:8] -> [funA:8] <- TOP

Memory: 16/28 B

funC called

Stack: [funA:8] -> [funA:8] -> [funC:12] <- TOP

Memory: 28/28 B

funC finished

Stack: [funA:8] -> [funA:8] <- TOP

Memory: 16/28 B

funA finished

Stack: [funA:8] <- TOP

Memory: 8/28 B

funB called

Stack: [funA:8] -> [funB:8] <- TOP

Memory: 16/28 B

funB finished

Stack: [funA:8] <- TOP

Memory: 8/28 B

funC called

Stack: [funA:8] -> [funC:12] <- TOP

Memory: 20/28 B

funC finished

Stack: [funA:8] <- TOP

Memory: 8/28 B

funA finished

Stack: EMPTY

Memory: 0/28 B

Execution Summary

Total call attempts: 6

Successful calls: 6

Skipped due to stack overflow: 0

Maximum stack depth reached: 3

Maximum stack memory used: 28 B

Total stack capacity: 28 B

Most frequently called function: funA

===== Test Case 2 =====

main called

Stack: [main:8] <- TOP

Memory: 8/20 B

main called

Stack: [main:8] -> [main:8] <- TOP

Memory: 16/20 B

Error: Stack overflow while calling work

main finished

Stack: [main:8] <- TOP

Memory: 8/20 B

work called

Stack: [main:8] -> [work:8] <- TOP

Memory: 16/20 B

Error: Stack overflow while calling help

work finished

Stack: [main:8] <- TOP

Memory: 8/20 B

main finished

Stack: EMPTY

Memory: 0/20 B

Execution Summary

Total call attempts: 5

Successful calls: 3

Skipped due to stack overflow: 2
Maximum stack depth reached: 2
Maximum stack memory used: 16 B
Total stack capacity: 20 B
Most frequently called function: main

===== Test Case 3 =====

Error: Circular Dependency: funA -> funB -> funC -> funA

===== Test Case 4 =====

Error: Undefined function missing called by start

===== Test Case 5 =====

Error: Duplicate definition of task

Question 2

1. Scenario and Objective

During lexical analysis, a compiler must distinguish structural delimiters from characters that merely appear inside strings, character literals, comments, or preprocessing directives. You are required to develop a C++ Source Code Bracket Analyzer that uses a stack to validate the structural brackets (), {}, and [] in source-code test cases.

The analyzer must read multiple independent source-code test cases from a single text file, process each case character by character, maintain lexical state, report the first structural bracket error, and produce nesting statistics for valid cases.

2. Core Restrictions

- The bracket stack must be implemented using a dynamic array. Do not use built in stack.
- All test cases are read from a single text file.
- Process the source code character by character.
- Do not use regular expressions for bracket matching or remove strings/comments before analysis.
- General C++ syntax validation is not required; only the bracket and lexical rules stated below are considered.

3. Bracket Matching

Only brackets encountered in normal source code participate in stack operations. Push every opening bracket (, {, or [. A closing bracket), }, or] must match the most recent unmatched opening bracket.

Each stack entry must store the opening bracket together with the line number and column number at which it appeared.

4. Lexical States and Ignored Regions

The analyzer must maintain its current lexical state while scanning the file. At minimum, distinguish the following states:

- NORMAL CODE
- STRING LITERAL
- CHARACTER LITERAL
- SINGLE-LINE COMMENT
- MULTI-LINE COMMENT

Brackets appearing inside the following regions must be completely ignored:

`"string literals"`

`'character literals'`

`// single-line comments`

`/* multi-line comments */`

A single-line comment ends at the newline. A multi-line comment begins with `/*` and ends at the first `*/`. Nested multi-line comments are not supported.

5. Escaped Characters

An escaped quote must not terminate the current string or character literal. The analyzer must distinguish an escaped quote from the delimiter that actually ends the literal.

```
cout << "He said \"Hello [world]\"";\nchar c = '\\';
```

6. Preprocessor Directives

If the first non-space character of a source-code line is `#`, ignore that entire line. Brackets appearing on such a line must not affect the stack.

```
#define ACCESS(x) ((x)[0])
```

7. Error Handling

Report only the first structural bracket error in each test case. Once an error is detected, stop analyzing that test case and continue with the next test case.

7.1 Unexpected Closing Bracket

If a closing bracket is encountered while the stack is empty, report:

```
INVALID
```

```
Error at Line 4, Column 8: Unexpected ']'
```

7.2 Mismatched Closing Bracket

If a closing bracket does not match the opening bracket at the top of the stack, report the expected and actual bracket:

```
INVALID
```

```
Error at Line 7, Column 12: Expected ']' but found ')'
```

7.3 Unclosed Opening Bracket

If the end of a test case is reached while the stack is not empty, report the most recently opened unmatched bracket and its original position:

```
INVALID
```

```
Error: '[' opened at Line 3, Column 6 was never closed
```

8. Valid-Case Statistics

For a valid test case, display:

```
VALID
```

```
Maximum Nesting Depth: 4
```

```
Total Matched Pairs: 9
```

Maximum nesting depth is the largest stack size reached during the test case. Total matched pairs counts only structural brackets processed in NORMAL CODE.

9. Input File and Multiple Test Cases

You may use AI tools to implement this part. No questions will be asked regarding this functionality during the demo. However, this functionality is mandatory, and failure to implement it will result in negative marking.

Input must be read from a text file (for example, input.txt). A single file may contain multiple independent source-code test cases. Test cases are separated by a line containing exactly:

###

The separator is handled by the test-case reader and must not be interpreted as source code or as a preprocessor directive.

- Line numbering restarts from 1 for every test case.
- Column numbering starts from 1 on every new line.
- Blank lines inside a test case count toward line numbering.
- When ### is reached, reset the stack, lexical state, counters, and statistics before processing the next case.
- Output is displayed on the console; an output file is not required.

10. Required Processing Order Per Test Case

1. Read the complete source-code test case until ### or end-of-file.
2. Reset the stack, lexical state, line/column counters, matched-pair count, and maximum nesting depth.
3. Scan the test case character by character while maintaining the current lexical state.
4. Ignore brackets in strings, character literals, comments, and preprocessor-directive lines.
5. For brackets in NORMAL CODE, perform the required push/match/pop operations.
6. Stop at the first structural bracket error and print the required error message.
7. If no error occurs, check for any unmatched opening bracket at the end of the test case.
8. For a valid test case, print the maximum nesting depth and total matched pairs.

11. Sample Test Case

INPUT FILE:

```
int main() {
    string x = "[not a bracket]";
    char c = '}';
    if (arr[2] > 0) {
        // ignore this: })
        cout << "Value \"(test)\"";
    }
}
###
```

```

int main() {
    if (arr[2] > 0) {
        return 0;
    }
}

###
#define TEST(x) ((x)[0])
int main() {
    /* ignore [ { ( here */
    cout << "[[(ignored)]]";
    return 0;
}

```

OUTPUT:

===== Test Case 1 =====

VALID

Maximum Nesting Depth: 3

Total Matched Pairs: 5

===== Test Case 2 =====

INVALID

Error at Line 2, Column 14: Expected ']' but found '}'

===== Test Case 3 =====

VALID

Maximum Nesting Depth: 1

Total Matched Pairs: 2